Research Article

CUDA Optimization of Fused Causal Softmax for Transformer Attention

Arwin Karir
Educational GPU Systems Research Project

Abstract

Background: Causal scaled softmax is a reduction-heavy stage between the two matrix multiplications of transformer attention. **Objective:** This study asks how CUDA work decomposition, reduction strategy, warp communication, and launch configuration affect this operator, and how much isolated-kernel improvement transfers to complete attention. **Methods:** One evolving FP32 CUDA implementation progressed from one thread per row to one block per row with shared-memory trees and then compact warp-shuffle reductions. Correctness was compared with PyTorch, while CUDA events measured 100 samples after 25 warmups on an NVIDIA Tesla T4 for sequence lengths 128–2048. **Results:** All 88 structured softmax cases passed at rtol $= 10^{-5}$ and atol $= 10^{-6}$, with maximum absolute error 3.5763×10^{-7}. The warp kernel was 3.22–8.92× faster than the row-serial implementation, 1.07–1.91× faster than the shared-tree implementation, and 1.40–3.94× faster than equivalent PyTorch eager softmax. Explicit attention improved by 1.54–2.31×, but PyTorch scaled-dot-product attention remained faster at every tested shape. **Conclusion:** Work mapping and warp communication materially improved the isolated kernel, while unchanged matrix multiplications and broader attention fusion limited application-level gains.

Keywords: CUDA; GPU optimization; stable softmax; causal attention; transformer attention; kernel fusion; parallel reduction.

1. INTRODUCTION

Transformers organize sequence modeling around attention, which exposes parallel work across tokens while learning data-dependent relationships [1]. Scaled dot-product attention is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}} + M\right)V, \quad (1)$$

where d is the head dimension and M excludes invalid positions. In causal self-attention, a query at position i may attend only to keys at positions $j \leq i$. PyTorch exposes this operation through scaled-dot-product attention (SDPA) and can dispatch to optimized backends depending on the inputs and runtime [2].

Equation 1 contains two matrix multiplications around a reduction-heavy normalization. Softmax requires a maximum reduction for numerical stability, exponentiation, a denominator reduction, and final normalization. A framework composition also performs scaling and causal masking as separate tensor work. This structure makes causal softmax a compact case study in GPU work decomposition, coalesced memory access, synchronization, shared memory, register-local partials, and warp communication.

This work studies one primary CUDA source file through Git history rather than maintaining unrelated "naive" and "optimized" implementations. The research question is:

> How do GPU work decomposition, parallel reductions, warp-level communication, and launch configuration affect fused causal scaled-softmax performance, and how much do those kernel-level optimizations translate into end-to-end transformer attention performance?

The contributions are: (1) an inspectable fused causal scaled-softmax operator; (2) commit-linked comparisons of row-serial, shared-tree, and warp-reduction designs; (3) controlled launch and framework comparisons; (4) integration into explicit attention with a production SDPA baseline; and (5) a reproducible evidence archive containing raw samples, environment metadata, figures, profiler exports, and the executed experiment notebook.

2. BACKGROUND AND RELATED WORK

2.1. Stable Causal Softmax

For logits $x_1, \ldots, x_n$, numerically stable softmax subtracts the row maximum m:

$$p_i = \frac{\exp(x_i - m)}{\sum_j \exp(x_j - m)}, \qquad m = \max_j x_j. \quad (2)$$

Multiplying numerator and denominator by $\exp(-m)$ shows that the probability is mathematically unchanged. Numerically, the largest exponential argument is zero and all others are non-positive, preventing large positive logits from overflowing the exponential. Stable log-sum-exp and

Corresponding author: Arwin Karir
Code and evidence: `https://github.com/Arwin-K/cuda-attention-softmax`

softmax evaluation has been studied directly [3], and online normalization work demonstrates how reorganization can reduce memory access [4].

Floating-point addition is not associative, so different parallel reduction orders need not match bit-for-bit. CUDA and PyTorch document the resulting rounding and reproducibility constraints [5]–[8]. Accordingly, this study fixes error tolerances before performance evaluation rather than requiring exact equality or weakening tolerances after observing results.

2.2. CUDA Reductions and Warp Communication

CUDA groups threads into blocks and blocks into a grid. Threads in a block can cooperate through shared memory and block-wide barriers; hardware executes threads in warps, conventionally 32 lanes on the tested architecture [9]. A block reduction can combine one partial per thread through a shared-memory tree. Warp shuffle operations instead exchange register values within a warp, avoiding a shared-memory write–barrier–read sequence at each intra-warp reduction step [10]. A compact two-level design can reduce within each warp, store one result per warp in shared memory, and use the first warp for the final block result.

Performance does not follow from occupancy or thread count alone. Block size, register use, shared memory, coordination, and row length interact, so launch configuration must be measured rather than inferred [6], [11].

2.3. Fusion and Attention Systems

Kernel fusion can reduce intermediate storage and launch overhead by retaining related work within one kernel. FlashAttention showed that IO-aware tiling can substantially improve exact full attention by reducing high-bandwidth-memory traffic [12]; FlashAttention-2 further improved work partitioning and parallelism [13]. The present operator is deliberately narrower: it fuses the scaled causal softmax stage but leaves QK^T and PV to PyTorch. This boundary makes the difference between a microkernel result and a full-attention result explicit.

3. METHODOLOGY

3.1. Operation Contract

For $Q, K, V \in \mathbb{R}^{B\times H\times S\times D}$, the score tensor is flattened from $[B, H, S, S]$ to $[R, S]$, where $R = BHS$. For row r, the query position is

$$q_r = r \bmod S. \tag{3}$$

Only columns $j \le q_r$ are allowed. Given a finite positive scale α, the fused operator computes

$$m_r = \max_{j\le q_r}(\alpha s_{r,j}), \tag{4}$$

$$z_r = \sum_{k\le q_r} \exp(\alpha s_{r,k} - m_r), \tag{5}$$

$$p_{r,j} = \begin{cases} \exp(\alpha s_{r,j} - m_r)/z_r, & j \le q_r, \\ 0, & j > q_r. \end{cases} \tag{6}$$

The output preserves shape, dtype, and device. The implementation accepts contiguous two-dimensional FP32 CUDA tensors and implements forward execution only.

3.2. Hypotheses

Five hypotheses were evaluated after evidence collection:

1. row-serial work scales poorly as row length increases;
2. one block per row improves sufficiently long rows despite coordination;
3. warp reductions improve on full shared-memory trees;
4. the preferred block size varies with sequence length; and
5. isolated-softmax speedup usually exceeds complete-attention speedup.

3.3. Correctness Protocol

The custom output was compared with an independent PyTorch composition at rtol = 10^{-5} and atol = 10^{-6}. Sequence lengths were 31, 32, 33, 63, 64, 127, 128, 255, 511, 768, and 1023. Eight input families covered normal random values; random values multiplied by 10, 100, and 1000; zeros; equal values; one dominant positive value; and one dominant negative value. Tests checked numerical closeness, exact causal zeros, approximate row sums of one, finiteness, shape, dtype, and device. Complete attention was also compared with explicit PyTorch attention and SDPA.

3.4. Benchmark Protocol

The isolated-softmax experiment used FP32, $R = 8S$, scale $1/\sqrt{64}$, and $S \in \{128, 255, 512, 768, 1024, 1536, 2048\}$. Attention used batch 1, eight heads, and head dimension 64. Inputs, masks, and allocations were outside the timed interval. Each case used 25 untimed warmups followed by 100 CUDA-event samples. The median was the primary latency statistic, with the 25th and 75th percentiles retained. CUDA synchronization ensured that device completion was included. The initial `torch.compile` call was excluded from steady-state latency and recorded as a separate compile warmup [14]–[17].

Every result row records its Git revision, implementation, shape, dtype, warmups, iterations, device, compute capability, PyTorch version, CUDA version, and timestamp. Raw samples are preserved rather than replaced by summaries.

4. CUDA IMPLEMENTATION EVOLUTION

4.1. Row-Serial Baseline

The first complete design assigned one softmax row to one CUDA thread. That thread scanned allowed columns for the maximum, scanned again for the stable exponential sum, and normalized the row. The mapping was simple and provided a correct baseline, but exposed no parallelism inside a row.

4.2. Block-Parallel Shared Trees

The second design assigned one block to one row. Thread t visited columns $t + k$ `blockDim.x`, giving neighboring threads neighboring initial addresses while supporting arbitrary row lengths. Each thread accumulated register-local maximum and denominator partials. Full shared-memory trees then combined those partials, with block barriers protecting producer–consumer boundaries. A second strided pass normalized output columns in parallel.

4.3. Compact Warp Reductions

The final design used `__shfl_down_sync` for maximum and sum reductions within each warp. Each warp leader wrote one value to a compact shared array; after one block barrier, the first warp completed the cross-warp reduction. For a 128-thread block, four warp partials require 16 bytes of dynamic shared memory. Scaling, the causal predicate, stable maximum subtraction, exponentiation, both reductions, and normalization remain fused in one kernel.

Historical algorithms were recovered from commits `8f07d762` (row serial), `f9420de0` (shared tree), and `a3736910` (warp reduction). The historical rebuild applied a recorded header-only include needed by CUDA 12.8; no algorithm, workload, or tolerance was changed.

5. EXPERIMENTAL SETUP

Measurements were collected in one Google Colab session. Table 1 summarizes the captured environment. The NVIDIA driver advertised CUDA 13.0 compatibility, while the installed toolkit and PyTorch build used CUDA 12.8. Current framework and attention benchmarks correspond to clean commit `ca87722a00ebf585cd788c67949e7c0b32dca788`.

Table 1. Measured execution environment.

Component	Recorded value
GPU	NVIDIA Tesla T4, compute capability 7.5, 15.6 GB
Host	Google Colab; Linux 6.6.122, x86-64
Python	3.13.15
PyTorch	2.11.0+cu128
Toolkit	NVCC 12.8.93
Driver	580.82.07; CUDA compatibility 13.0
C++ compiler	GCC 11.4.0
Timing	25 warmups; 100 CUDA-event samples

Table 2. Correctness evidence summary at fixed FP32 tolerances.

Check	Softmax	Attention
Cases passed	88/88	7/7
Maximum absolute error	3.5763×10^{-7}	2.9802×10^{-7}
Maximum row-sum error	3.5763×10^{-7}	—
Masked future values	all zero	causal outputs passed
Unexpected NaN/Inf	none	none

6. RESULTS

6.1. Correctness

The CUDA test stage reported 70 passes. The structured softmax matrix contained 88 comparisons (11 lengths times 8 input families), all of which passed. The maximum absolute error was 3.5762787×10^{-7}, maximum relative error was 4.6973432×10^{-7}, and maximum row-sum error was 3.5762787×10^{-7}. Every masked future probability was exactly zero, and no unexpected NaN or infinity appeared. Table 2 summarizes the accepted evidence.

All seven complete-attention comparisons passed. Maximum absolute output error was 2.9802322×10^{-7} for custom explicit attention versus explicit PyTorch and 7.1525574×10^{-7} for SDPA versus the same reference.

6.2. Historical Kernel Evolution

Figure 1 shows the matched historical comparison. The warp kernel was 3.22–8.92× faster than row serial and 1.07–1.91× faster than the shared-tree design across all tested lengths. The larger row-serial gap combines work-decomposition and reduction improvements; the shared-tree comparison more closely isolates the communication change after block-per-row work was already present.

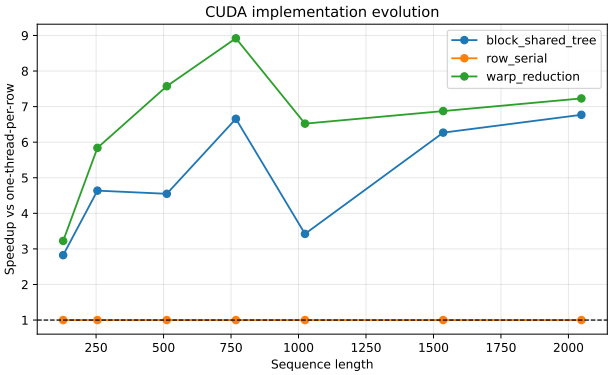

Figure 1. Historical kernel speedup from matched raw CUDA-event samples. Row serial (`8f07d762`) is the reference for the plotted ratios.

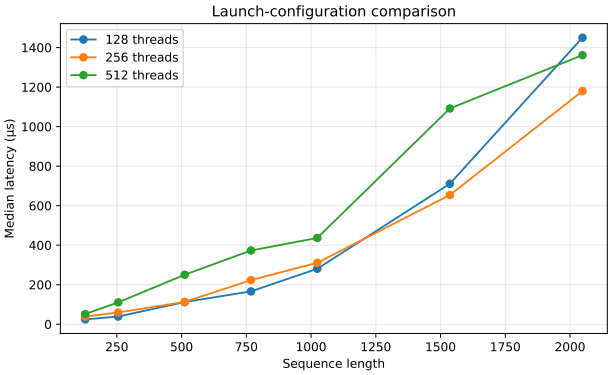

Figure 2. Launch-configuration median latency on the Tesla T4. Only block size changes within this comparison.

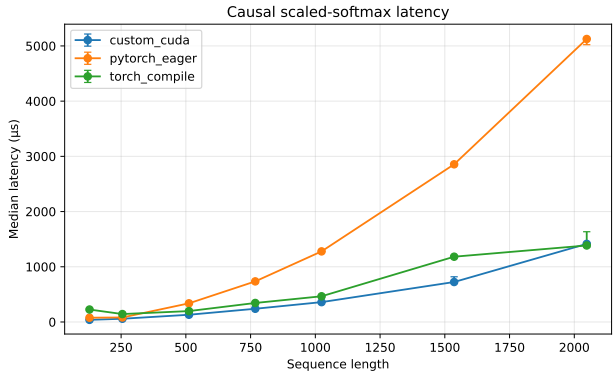

Figure 3. Median softmax latency for custom CUDA and framework baselines.

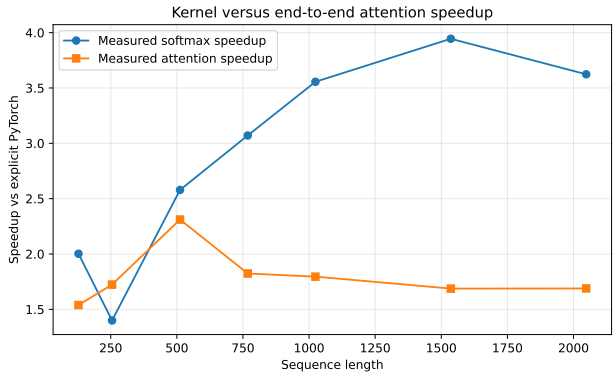

Figure 4. Isolated-softmax and explicit-attention speedup over their respective eager PyTorch baselines.

6.3. Launch Configuration

The 128-thread block had the lowest median at lengths 128, 255, 512, 768, and 1024. A 256-thread block won at 1536 and 2048; 512 threads won no tested shape. The predefined aggregate rule normalized latency by the fastest configuration at each length and selected the block size with the lowest median relative latency. It selected 128 threads with score 1.000, compared with 1.107 for 256 and 2.100 for 512. Figure 2 demonstrates that the per-shape winner is not universal.

6.4. Framework Softmax Comparison

Table 3 and Figure 3 compare equivalent scale–mask–softmax work. The custom operator beat eager PyTorch at all seven lengths by 1.40–3.94×. It beat steady-state `torch.compile` at six lengths; at $S = 2048$, compiled PyTorch was approximately 2.1% faster.

6.5. Complete Attention

The explicit custom path computes QK^T, invokes the custom softmax, and then computes PV. It was 1.54–2.31× faster than explicit eager attention. Kernel speedup exceeded attention speedup at six of seven lengths, consistent with the surrounding matrix multiplications limiting the benefit of optimizing only softmax. Production SDPA remained 1.25–2.53× faster than the explicit custom path at every length. This is a mathematically comparable output baseline, but not an implementation-equivalent fusion boundary: SDPA can fuse a larger portion of attention.

6.6. Profiling

Measured. At $S = 512$, PyTorch Profiler attributed 110.207 μs of CUDA time to the fused kernel. Named device totals were 505.565 μs for custom explicit attention, 1095.195 μs for explicit eager attention, and 408.222 μs for SDPA. Nsight Compute 2025.1.1 captured four target launches and reported 128 threads per block, 24 registers per thread, zero static shared memory, and 16 bytes of dynamic shared memory. Kernel durations were 106.976–107.584 μs, with 48.33–48.79% peak DRAM throughput and 56.72–57.00% peak SM throughput [18], [19].

Interpretation. The dynamic shared-memory footprint agrees with four warp partials for a 128-thread block. The throughput percentages do not prove a single memory or compute bottleneck. Profiler captures use different conditions from repeated benchmark medians and are not substituted for them.

Table 3. Median fused causal softmax latency (μs) and custom speedup over equivalent eager PyTorch. Each median summarizes 100 samples.

S	128	255	512	768	1024	1536	2048
Custom CUDA	38.752	58.544	131.040	239.440	359.568	723.952	1414.480
PyTorch eager	77.600	81.920	337.920	735.264	1278.592	2855.840	5125.488
`torch.compile`	225.664	143.456	197.392	344.352	465.424	1182.416	1385.760
Custom/eager	2.00×	1.40×	2.58×	3.07×	3.56×	3.94×	3.62×

Table 4. Median complete-attention latency (μs) and explicit custom speedup over explicit eager attention.

S	128	255	512	768	1024	1536	2048
Custom explicit	129.008	204.288	323.584	587.312	1014.016	2438.960	4198.768
Explicit eager	198.544	352.128	747.888	1071.088	1820.672	4115.824	7090.240
PyTorch SDPA	84.000	163.600	245.808	384.864	543.744	1032.400	1660.576
Custom/eager	1.54×	1.72×	2.31×	1.82×	1.80×	1.69×	1.69×

Table 5. PyTorch Profiler device time at $S = 512$. Path totals and the fused kernel row describe different scopes and should not be added.

Profiled scope	Device time (μs)
Custom explicit-attention total	505.565
Explicit eager-attention total	1095.195
PyTorch SDPA total	408.222
Fused causal-softmax kernel	110.207

Table 6. Hypothesis status from the measured T4 experiment.

ID	Status	Evidence boundary
H1	Partially supported	Row-serial latency grew 58.98× versus 24.61× for block/shared tree.
H2	Supported	Block-per-row improved the tested long shapes by 3.42–6.77×.
H3	Supported	Warp was faster than shared tree at all seven lengths.
H4	Supported	128 and 256 threads each won at least one shape.
H5	Partially supported	Kernel speedup exceeded attention speedup for 6/7 lengths.

7. DISCUSSION

7.1. Hypothesis Evaluation

Table 6 summarizes the evidence-bounded hypothesis outcome. The row-serial growth trend was only partially supported because latency trends do not alone prove the causal mechanism. Block-per-row work was supported at the tested long shapes. Warp reduction and shape-dependent launch choice were supported on the T4. The kernel-to-attention prediction was partially supported because the kernel speedup exceeded attention speedup at six, not all seven, matched lengths.

7.2. Microkernel Versus Application Performance

The historical comparison shows that work decomposition produced the largest gain: assigning a block rather than a thread to a row exposed intra-row parallelism. Warp communication then produced a smaller but consistently positive improvement over a block design that was already parallel.

Application improvement was smaller because the custom kernel changes only one stage. If fraction f of baseline attention time is accelerated by factor s, Amdahl's Law gives

$$S_{\text{total}} = \frac{1}{(1-f) + f/s}. \tag{7}$$

The measured relationship broadly follows this limit, although the simple model does not reproduce every short-shape latency because launch overhead, backend selection, and interactions among operations also matter [20].

SDPA's consistent advantage is a central result. The custom softmax improves an explicit attention composition, but a production backend can optimize and fuse across a broader boundary. A fast isolated component is therefore useful evidence, not proof of a superior end-to-end attention implementation.

8. LIMITATIONS

All quantitative performance findings come from one Tesla T4 in one Colab session. They do not establish run-to-run variability or transfer to Ampere, Ada, Hopper, or other architectures. Managed infrastructure does not fully control clock state, contention, or future software images. Quartiles describe within-session samples; they are not confidence intervals across machines or sessions.

The operator implements contiguous FP32 forward

causal scaled softmax. It does not provide backward/autograd support, dropout, arbitrary or padding masks, variable valid lengths, or mixed precision. The flattened causal contract assumes scores derived from $[B, H, S, S]$. Historical kernels required a recorded header include for CUDA 12.8, so the rebuilds are algorithmically matched but not byte-identical to their original source snapshots.

Framework comparisons were designed to perform equivalent scale, mask, and softmax work, but framework internals can evolve. Compilation startup was excluded from steady-state `torch.compile` latency and should be evaluated separately for short-lived workloads. SDPA is an output-equivalent production baseline, not an equal-fusion-boundary microkernel baseline.

Nsight profiled only one sequence length and four launches. Peak DRAM and SM throughput percentages do not uniquely identify a bottleneck. Matched profiler metrics for historical kernels and independent sessions remain future work.

9. CONCLUSION AND FUTURE WORK

On the measured Tesla T4 workload, changing from row-serial to block-per-row work delivered the largest kernel improvement, compact warp communication improved further on full shared-memory trees, and block-size preference varied with shape. The custom operator substantially outperformed equivalent eager softmax and accelerated explicit attention, but did not outperform production SDPA. The central systems result is that GPU mapping and communication choices matter locally, while end-to-end value depends on the fraction and fusion boundary of the larger application.

Future work should repeat the complete experiment across independent sessions and on a newer architecture; profile shared-tree and warp kernels under matched conditions; evaluate shape-aware launch dispatch; add FP16/BF16 accumulation analysis and fixed error criteria; add backward/autograd support; vary batch, head count, head dimension, and masking; and compare specialized library operators under explicitly equivalent work.

10. DECLARATIONS AND ARTIFACT AVAILABILITY

Author contribution (CRediT): Arwin Karir: Conceptualization, Methodology, Software, Validation, Formal Analysis, Investigation, Data Curation, Visualization, Writing—Original Draft, and Writing—Review and Editing.

Conflict of interest: No conflict of interest is recorded for this independent educational research project.

Funding: No external funding is recorded for this project. The measured experiment used a Google Colab Tesla T4 runtime.

Data and code availability: Source, tests, notebook, raw CUDA-event samples, summaries, profiler exports, environment metadata, and generated figures are available in the public GitHub repository. The preserved evidence is in run `2026-08-23_tesla-t4_ca87722`; its manifest records the full source revision `ca87722a`.

AI-assistance disclosure: OpenAI Codex was used to assist with manuscript restructuring, LaTeX formatting, copyediting, and repository build automation. It is not an author. The human author remains responsible for verifying the source code, citations, measurements, interpretations, and final submitted manuscript.

Ethics statement: This computer-systems study did not involve human participants, personal data, animals, or biological specimens.

Acknowledgments: The project relies on the open-source PyTorch ecosystem and NVIDIA CUDA development and profiling tools.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems 30*, 2017. [Online]. Available: `https://papers.nips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html`.

[2] PyTorch Contributors. "Torch.nn.functional.scaled_dot_product_attention." (2026), [Online]. Available: `https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.scaled_dot_product_attention.html` (visited on 08/17/2026).

[3] P. Blanchard, D. J. Higham, and N. J. Higham, "Accurate computation of the log-sum-exp and softmax functions," *arXiv preprint arXiv:1909.03469*, 2019. DOI: `10.48550/arXiv.1909.03469`. [Online]. Available: `https://arxiv.org/abs/1909.03469`.

[4] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," *arXiv preprint arXiv:1805.02867*, 2018. DOI: `10.48550/arXiv.1805.02867`. [Online]. Available: `https://arxiv.org/abs/1805.02867`.

[5] NVIDIA Corporation. "Floating-point computation — CUDA programming guide." (2026), [Online]. Available: `https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/mathematical-functions.html` (visited on 08/17/2026).

[6] NVIDIA Corporation. "CUDA best practices guide." (2026), [Online]. Available: `https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html` (visited on 08/17/2026).

[7] PyTorch Contributors. "Numerical accuracy." (2026), [Online]. Available: `https://docs.pytorch.org/docs/stable/notes/numerical_accuracy.html` (visited on 08/17/2026).

[8] PyTorch Contributors. "Reproducibility." (2026), [Online]. Available: `https://docs.pytorch.org/docs/stable/notes/randomness.html` (visited on 08/17/2026).

[9] NVIDIA Corporation. "CUDA programming guide." (2026), [Online]. Available: `https://docs.nvidia.com/cuda/cuda-programming-guide/index.html` (visited on 08/17/2026).

[10] NVIDIA Corporation. "Warp shuffle functions — CUDA programming guide." (2026), [Online]. Available: https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-extensions.html (visited on 08/17/2026).

[11] NVIDIA Corporation. "Occupancy — CUDA runtime api." (2026), [Online]. Available: https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__OCCUPANCY.html (visited on 08/17/2026).

[12] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems 35*, 2022. DOI: 10.52202/068431-1189. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html.

[13] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=mZn2Xyh9Ec.

[14] NVIDIA Corporation. "Event management — CUDA runtime api." (2026), [Online]. Available: https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__EVENT.html (visited on 08/17/2026).

[15] PyTorch Contributors. "Torch.cuda.event." (2026), [Online]. Available: https://docs.pytorch.org/docs/stable/generated/torch.cuda.Event.html (visited on 08/17/2026).

[16] PyTorch Contributors. "Torch.cuda.synchronize." (2026), [Online]. Available: https://docs.pytorch.org/docs/stable/generated/torch.cuda.synchronize.html (visited on 08/17/2026).

[17] PyTorch Contributors. "Torch.compile." (2026), [Online]. Available: https://docs.pytorch.org/docs/stable/generated/torch.compile.html (visited on 08/17/2026).

[18] PyTorch Contributors. "PyTorch profiler." (2026), [Online]. Available: https://docs.pytorch.org/tutorials/recipes/recipes/profiler_recipe.html (visited on 08/17/2026).

[19] NVIDIA Corporation. "NVIDIA Nsight Compute profiling guide." (2026), [Online]. Available: https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html (visited on 08/17/2026).

[20] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference*, 1967, pp. 483–485. DOI: 10.1145/1465482.1465560. [Online]. Available: https://dl.acm.org/doi/10.1145/1465482.1465560.